Principal Component Analysis and Its Mathematical Justification Through the Eckart–Young Theorem

A Step-by-Step Development from Geometric Intuition to Optimal Low-Rank Approximation

Biajid Chowdhury

October 10, 2026

Abstract

In recent years, Principal Component Analysis (PCA) has gained enormous popularity in data science, machine learning, and statistical analysis. Despite its widespread application, however, its underlying mathematical justification is often given surprisingly little attention, particularly in introductory data science curricula. Instead, PCA is frequently introduced as a black-box computational tool: subtract the mean, compute the Singular Value Decomposition (SVD), retain a few dominant components, and proceed with the transformed data. Although this approach is convenient, it leaves a fundamental mathematical question unanswered: *Why should these particular components provide the best possible lower-dimensional representation of the original data?*

Fortunately, with relatively modest mathematical background and some geometric intuition, one can appreciate the remarkable beauty of PCA and understand why it works. Such an understanding not only removes the mystery behind the algorithm but also allows one to apply it more intelligently and effectively.

In this note, we develop Principal Component Analysis step-by-step, beginning with the geometric interpretation of data projection, variance, and Singular Value Decomposition. We then explain how principal components are obtained, how they transform the original data, and how a lower-dimensional representation can be used to reconstruct an approximation of the original dataset. Finally, we establish the mathematical optimality of this reconstruction through the Eckart–Young–Mirsky theorem, examining both the Frobenius and spectral norms. Our objective is to bridge the gap between the computational recipe commonly taught in data science and the elegant linear algebra that justifies it.

Contents

1 Introduction **3**

2 Prerequisite Mathematical Background **4**

3 A Brief Review of Singular Value Decomposition **5**

3.1 Row Space and Column Space 5

3.2 The Singular Value Decomposition Theorem . . . 6
3.3 How Are the Singular Vectors Obtained? . . . 7
3.4 The Reduced Singular Value Decomposition . . . 9
3.5 The Geometric Meaning of SVD . . . 9
3.6 SVD and Orthogonal Projections . . . 10
3.7 From the Row Space to the Column Space . . . 11
3.8 Why Is This Relevant to PCA? . . . 12

4 Geometric Motivation for Principal Component Analysis 13

4.1 Representing a Dataset Geometrically . . . 13
4.2 What Does Variance Mean Geometrically? . . . 14
4.3 The Covariance Matrix . . . 16
4.4 The Principal Component Analysis Procedure . . . 22
4.5 What Have We Actually Accomplished? . . . 29
4.6 The Connection with Reconstruction Error . . . 30
4.7 The Optimality Question . . . 31

5 Mathematical Justification of Optimal Low-Rank Approximation: The Eckart–Young Theorem 33

5.1 The Approximation Problem and Matrix Norms . . . 33
5.2 Statement of the Eckart–Young Theorem . . . 34
5.3 Proof Under the Frobenius Norm . . . 35
5.4 Proof Under the Spectral Norm . . . 40
5.5 What Do These Results Tell Us About PCA? . . . 42

6 Numerical Implementation of PCA Using Python 43

6.1 Implementation Using a Small Dataset . . . 43
6.2 Implementation Using a Real Dataset . . . 43
6.3 Python Source Code . . . 44

7 Conclusion and Future Directions 49

1 Introduction

In modern data science and machine learning, one frequently encounters datasets involving hundreds, thousands, or even millions of variables. Although such high-dimensional data may contain valuable information, working directly with all these variables is not always computationally efficient or mathematically relevant. Furthermore, many of these variables may be strongly correlated, meaning that a considerable amount of the variation in the data can often be represented using substantially fewer dimensions.

This naturally raises an interesting question: *Can we represent a high-dimensional dataset using only a few carefully chosen directions, while preserving as much of its original variation as possible?*

Principal Component Analysis (PCA) provides an elegant mathematical answer to this question. The central idea is remarkably simple: rather than describing observations using the original coordinate axes, we construct a new set of mutually orthogonal directions along which the data exhibits the greatest variation. By retaining only the most significant of these directions, we obtain a lower-dimensional representation of the original dataset.

At first glance, the computational procedure appears almost deceptively simple. We begin by subtracting the mean of each variable, perform a Singular Value Decomposition (SVD) of the centered data matrix, retain a few singular vectors corresponding to the largest singular values, and project the observations onto the subspace spanned by these vectors.

But why should this procedure work?

More precisely, why should the singular vectors associated with the largest singular values identify the directions of maximum variation? What exactly happens to the original observations when we project them onto these directions? And perhaps most importantly, if we reconstruct the original dataset from its lower-dimensional representation, can we guarantee that the resulting approximation is the best possible one?

These questions reveal that PCA is much more than a convenient computational algorithm. Beneath its apparently simple implementation lies a beautiful connection among orthogonal projections, eigenvalue problems, matrix norms, and low-rank matrix approximation.

The mathematical foundation of this connection is provided by two closely related optimization principles. First, the principal directions maximize the variance captured by an orthogonal projection onto a subspace of prescribed dimension. Second, the same subspace minimizes the squared reconstruction error when the projected observations are mapped back into the original feature space.

The second property is closely connected to the celebrated Eckart–Young theorem, which establishes the optimality of truncated Singular Value Decomposition among all matrices of a prescribed maximum rank. In particular, the theorem shows that retaining the largest singular values produces an approximation with the smallest possible error, measured in either the Frobenius norm or the spectral norm.

In this note, our objective is not merely to present another recipe for performing PCA. Rather, we wish to develop the mathematical ideas that make the procedure work, beginning with familiar concepts from elementary linear algebra and gradually arriving at its rigorous justification.

We begin with a geometric discussion of high-dimensional data, orthogonal projections, and the meaning of variance along a given direction. We then formulate the problem of finding directions of maximum variance and establish its connection to the eigenvectors of the covariance matrix.

Next, we revisit Singular Value Decomposition from the perspective of PCA. We explain the geometric significance of the left and right singular vectors, the role of singular values, and how the principal components arise naturally from this decomposition.

Having established these ideas, we examine the process of dimensionality reduction and reconstruction. In particular, we investigate what information is retained, what information is discarded, and how the reconstruction error can be measured mathematically.

Finally, we state and prove the Eckart–Young theorem for both the Frobenius norm and the spectral norm. These proofs allow us to establish precisely why the truncated SVD provides an optimal low-rank approximation, rather than simply accepting its optimality as a computational fact.

Throughout the discussion, we emphasize geometric intuition alongside mathematical rigor. Whenever possible, abstract arguments will be accompanied by concrete examples, and the theoretical results will ultimately be verified through numerical experiments using Python.

Our hope is that, by the end of this note, PCA will no longer appear as an obscure black-box tool, but rather as a natural consequence of some of the most fundamental and elegant ideas in scientific computation and numerical linear algebra.

2 Prerequisite Mathematical Background

The primary objective of this note is to develop the mathematical justification of Principal Component Analysis rather than to introduce the elementary concepts of linear algebra from the beginning. Accordingly, we assume that the reader has some familiarity with the fundamental ideas ordinarily covered in an undergraduate linear algebra course.

In particular, the reader should be comfortable with the following concepts:

- **Vector spaces and subspaces:** Linear independence, basis, dimension, rank, null space, and the rank-nullity theorem.
- **Inner products and orthogonality:** The Euclidean inner product, orthonormal bases, orthogonal matrices, and orthogonal projections onto subspaces.
- **Eigenvalues and eigenvectors:** Eigenvalue decomposition, diagonalization, symmetric matrices, and the spectral theorem for real symmetric matrices.
- **Singular Value Decomposition (SVD):** The factorization

  $$A = U\Sigma V^T,$$

  including the meaning of singular values, left and right singular vectors, and their relationship to the eigenvalues and eigenvectors of AA^T and A^TA.
- **Basic matrix operations:** Matrix multiplication, transposition, matrix-vector multiplication, and elementary properties of matrix norms.

Readers who are unfamiliar with these topics, or who have studied them previously but would benefit from a review, are encouraged to consult Gilbert Strang's *Linear Algebra and Its Applications* before proceeding. In particular, the discussions of orthogonality, projections, eigenvalues, symmetric matrices, and Singular Value Decomposition provide an excellent foundation for understanding the material developed here.

Nevertheless, we do not expect the reader to be familiar with Principal Component Analysis, the Eckart–Young theorem, or the optimization problems associated with low-rank approximation. These topics will be developed carefully in the following sections.

We will also introduce the Frobenius and spectral norms explicitly and explain their geometric significance before using them in the proofs. Similarly, the necessary statistical concepts, particularly variance and covariance, will be reviewed in the context of PCA.

Although some of our arguments will involve optimization, no advanced background in optimization theory is assumed. The required results will be motivated and established using elementary linear algebra whenever possible.

Finally, we emphasize that this note is intended to be read with pencil and paper. The reader is encouraged to verify the matrix identities, work through the examples, and examine the geometric meaning of each transformation. Our goal is not simply to arrive at the correct formulas, but to understand why those formulas must take the form they do.

3 A Brief Review of Singular Value Decomposition

Before proceeding to the geometric and mathematical development of Principal Component Analysis, it is worthwhile to briefly revisit the Singular Value Decomposition (SVD) of a matrix.

We do not intend to provide a complete treatment of SVD here, as this is ordinarily covered in undergraduate linear algebra courses. Instead, we shall recall its fundamental properties, particularly the relationship between singular vectors, the row space, the column space, and orthogonal projections.

These ideas will play a central role in our subsequent development of PCA.

3.1 Row Space and Column Space

Let $A \in \mathbb{R}^{m \times n}$ be an arbitrary real matrix.

There are two important vector spaces naturally associated with A.

First, consider the columns of A as vectors in $\mathbb{R}^m$. Their linear combinations form the *column space* of A, denoted by

$$\mathrm{Col}(A) = \mathrm{span}\{a_1, a_2, \ldots, a_n\},$$

where a_j denotes the jth column of A.

In other words, the column space consists of all vectors that can be obtained by taking linear combinations of the columns of A.

Similarly, we may consider each row of A as a vector in $\mathbb{R}^n$. The linear combinations of these row vectors form the *row space* of A, denoted by

$$\mathrm{Row}(A) = \mathrm{Col}(A^T).$$

Thus, the column space is a subspace of $\mathbb{R}^m$, whereas the row space is a subspace of $\mathbb{R}^n$.

Although these two spaces generally belong to different Euclidean spaces, an important theorem from elementary linear algebra tells us that they have the same dimension:

$$\boxed{\dim \mathrm{Col}(A) = \dim \mathrm{Row}(A) = \mathrm{rank}(A).}$$

If $\mathrm{rank}(A) = r$, both spaces therefore admit bases consisting of r linearly independent vectors.

There is, however, no reason to insist that these bases consist of the original rows or columns of A.

Indeed, it is often much more convenient to construct orthonormal bases for these spaces. Singular Value Decomposition provides precisely such bases, together with an additional and rather remarkable structure.

3.2 The Singular Value Decomposition Theorem

Theorem 3.1 (Singular Value Decomposition)**.** *For every real matrix $A \in \mathbb{R}^{m\times n}$, there exist orthogonal matrices*

$$U \in \mathbb{R}^{m\times m}, \qquad V \in \mathbb{R}^{n\times n},$$

and a rectangular diagonal matrix

$$\Sigma \in \mathbb{R}^{m\times n},$$

such that

$$\boxed{A = U\Sigma V^T.}$$

The matrices U and V satisfy

$$U^TU = UU^T = I_m, \qquad V^TV = VV^T = I_n.$$

The diagonal entries of Σ are nonnegative real numbers, called the singular values of A.

If $\text{rank}(A) = r$*, the positive singular values can be arranged in nonincreasing order:*

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

All remaining diagonal entries of Σ *are zero.*

The columns of U are called the *left singular vectors*, while the columns of V are called the *right singular vectors*.

We write

$$U = \begin{bmatrix} u_1 & u_2 & \cdots & u_m \end{bmatrix},$$

and

$$V = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}.$$

Since these two matrices are orthogonal, their respective columns form orthonormal bases of $\mathbb{R}^m$ and $\mathbb{R}^n$.

More importantly, the first r singular vectors provide orthonormal bases for the fundamental spaces associated with A:

$$\boxed{\text{Col}(A) = \text{span}\{u_1, \ldots, u_r\}}$$

and

$$\boxed{\text{Row}(A) = \text{span}\{v_1, \ldots, v_r\}.}$$

Thus, SVD identifies orthonormal bases for both the column space and the row space of an arbitrary matrix.

3.3 How Are the Singular Vectors Obtained?

The singular values and singular vectors are closely related to the eigenvalue decompositions of AA^T and A^TA.

Starting from

$$A = U\Sigma V^T,$$

we obtain

$$AA^T = U\Sigma\Sigma^T U^T,$$

and similarly,

$$A^T A = V\Sigma^T\Sigma V^T.$$

Therefore, the columns of U are eigenvectors of AA^T, while the columns of V are eigenvectors of $A^T A$.

The nonzero eigenvalues of these two symmetric matrices are identical, and are given by

$$\boxed{\lambda_i = \sigma_i^2.}$$

Consequently, the positive singular values are obtained by taking the square roots of the positive eigenvalues of either AA^T or $A^T A$:

$$\sigma_i = \sqrt{\lambda_i}.$$

For each positive singular value, the corresponding left and right singular vectors satisfy

$$\boxed{Av_i = \sigma_i u_i, \qquad A^T u_i = \sigma_i v_i.}$$

In particular, if v_i is a normalized eigenvector of $A^T A$ corresponding to a positive eigenvalue σ_i^2, then

$$\boxed{u_i = \frac{Av_i}{\sigma_i}.}$$

Conversely, if u_i is known, we can obtain the corresponding right singular vector through

$$\boxed{v_i = \frac{A^T u_i}{\sigma_i}.}$$

These formulas apply whenever $\sigma_i > 0$. The remaining singular vectors are chosen to complete the respective orthonormal bases.

We may therefore interpret the singular value decomposition as a carefully coordinated pair of eigenvalue decompositions.

It is worth emphasizing that singular vectors are not necessarily unique. Repeated singular values allow different choices of orthonormal bases within the corresponding singular subspaces. Nevertheless, the decomposition always exists.

3.4 The Reduced Singular Value Decomposition

Suppose $\operatorname{rank}(A) = r$.

Instead of retaining all the columns of U and V, we may retain only those corresponding to positive singular values.

Define

$$U_r = \begin{bmatrix} u_1 & \cdots & u_r \end{bmatrix} \in \mathbb{R}^{m \times r},$$

$$V_r = \begin{bmatrix} v_1 & \cdots & v_r \end{bmatrix} \in \mathbb{R}^{n \times r},$$

and

$$\Sigma_r = \operatorname{diag}(\sigma_1, \ldots, \sigma_r).$$

Then

$$\boxed{A = U_r \Sigma_r V_r^T.}$$

This is called the *reduced* or *compact* singular value decomposition.

Equivalently, we may express A as a sum of rank-one matrices:

$$\boxed{A = \sum_{i=1}^{r} \sigma_i u_i v_i^T.}$$

Each term $\sigma_i u_i v_i^T$ has rank one.

This representation will become particularly useful when we discuss low-rank approximation and the Eckart–Young theorem.

3.5 The Geometric Meaning of SVD

Let us now consider the matrix A as a linear transformation

$$A : \mathbb{R}^n \longrightarrow \mathbb{R}^m.$$

For an arbitrary vector $x \in \mathbb{R}^n$, we have

$$Ax = U \Sigma V^T x.$$

The transformation can be understood as three consecutive operations.

First, V^T expresses the input vector x in the orthonormal coordinate system determined by the right singular vectors.

Next, Σ scales the resulting coordinates by the corresponding singular values. Coordinates associated with zero singular values are eliminated.

Finally, U expresses the resulting vector in the output space $\mathbb{R}^m$ using the left singular vectors.

Thus, schematically,

$$\boxed{x \xrightarrow{V^T} V^T x \xrightarrow{\Sigma} \Sigma V^T x \xrightarrow{U} Ax.}$$

In particular, since

$$Av_i = \sigma_i u_i,$$

the transformation A takes a right singular direction v_i and maps it to the corresponding left singular direction u_i, scaled by σ_i.

Hence, the positive singular values describe the stretching factors along the corresponding singular directions.

Geometrically, SVD identifies orthogonal input directions, their associated stretching factors, and the corresponding orthogonal output directions.

3.6 SVD and Orthogonal Projections

We now return to the row space and column space of A.

Recall that the columns of V_r form an orthonormal basis of $\mathrm{Row}(A)$.

Therefore, for any $x \in \mathbb{R}^n$, its orthogonal projection onto the row space is

$$\boxed{P_{\mathrm{Row}(A)} x = V_r V_r^T x.}$$

Similarly, since the columns of U_r form an orthonormal basis of $\mathrm{Col}(A)$, the orthogonal projection of any $y \in \mathbb{R}^m$ onto the column space is

$$\boxed{P_{\mathrm{Col}(A)} y = U_r U_r^T y.}$$

An important distinction deserves attention here.

The matrix V_r^T does not itself perform the orthogonal projection onto the row space in the original ambient coordinates. Rather,

$$V_r^T x$$

provides the coordinates of the row-space projection relative to the orthonormal basis $\{v_1, \ldots, v_r\}$. Multiplying these coordinates by V_r reconstructs the projected vector in $\mathbb{R}^n$:

$$V_r(V_r^T x) = V_r V_r^T x.$$

The same interpretation applies to the column space:

$$U_r^T y$$

provides the coordinates of the column-space projection, whereas

$$U_r(U_r^T y) = U_r U_r^T y$$

produces the actual projected vector in $\mathbb{R}^m$.

Thus, we obtain the important identities

$$\boxed{\begin{aligned} P_{\mathrm{Row}(A)} &= V_r V_r^T, \\ P_{\mathrm{Col}(A)} &= U_r U_r^T. \end{aligned}}$$

These formulas are fundamental consequences of the geometry of SVD.

3.7 From the Row Space to the Column Space

There is another useful way to understand the transformation A.

The Fundamental Theorem of Linear Algebra tells us that

$$\mathbb{R}^n = \mathrm{Row}(A) \oplus \mathrm{Null}(A),$$

where the two subspaces are orthogonal.

Thus, any input vector $x \in \mathbb{R}^n$ can be decomposed uniquely as

$$x = x_{\mathrm{row}} + x_{\mathrm{null}},$$

where

$$x_{\mathrm{row}} \in \mathrm{Row}(A), \qquad x_{\mathrm{null}} \in \mathrm{Null}(A).$$

Applying A, we obtain

$$Ax = Ax_{\text{row}} + Ax_{\text{null}}.$$

But $Ax_{\text{null}} = 0$. Consequently,

$$\boxed{Ax = Ax_{\text{row}}.}$$

Therefore, the action of A depends only on the component of x lying in its row space.

The null-space component contributes nothing to the output.

Furthermore, the resulting vector necessarily belongs to the column space of A.

In this sense, A takes the relevant information from its row space, stretches it along the singular directions, and maps it into its column space.

Indeed, for the reduced SVD,

$$Ax = U_r \Sigma_r V_r^T x.$$

The matrix V_r^T extracts the row-space coordinates, Σ_r scales them, and U_r constructs the corresponding vector in the column space.

This provides a particularly useful geometric interpretation of the singular value decomposition.

3.8 Why Is This Relevant to PCA?

When A represents a dataset with features arranged along the rows and observations along the columns, each observation is a vector in $\mathbb{R}^m$.

Thus, the left singular vectors u_i belong to the same feature space as the original observations.

As we shall establish in the subsequent sections, these vectors identify the principal directions along which the centered data exhibits variation.

If we retain only the first k left singular vectors, we obtain

$$U_k = \begin{bmatrix} u_1 & \cdots & u_k \end{bmatrix}.$$

The matrix

$$\boxed{Z = U_k^T A}$$

then expresses each observation in terms of these k orthonormal directions.

We may reconstruct an approximation in the original feature space by

$$\boxed{\widehat{A} = U_k Z = U_k U_k^T A.}$$

At this stage, we have only described the algebraic meaning of this operation.

We have not yet established why the first k singular vectors should be preferred over other possible directions.

That question forms the mathematical heart of PCA, and we shall address it through geometric intuition, variance maximization, and eventually the Eckart–Young theorem.

4 Geometric Motivation for Principal Component Analysis

The fundamental idea of PCA is that a collection of high-dimensional observations may exhibit most of its variation along only a few directions. If we can identify these directions, we may be able to describe the dataset using substantially fewer coordinates without losing too much of its original structure.

But what exactly do we mean by a direction that captures the variation of a dataset? And how can we determine which direction is the most important?

We begin with a simple geometric interpretation.

4.1 Representing a Dataset Geometrically

Suppose we have n observations, each described by m features. We represent the dataset by

$$X = \begin{bmatrix} | & | & & | \\ x_1 & x_2 & \cdots & x_n \\ | & | & & | \end{bmatrix} \in \mathbb{R}^{m \times n},$$

where each column $x_j \in \mathbb{R}^m$ represents one observation, and each row corresponds to a particular feature.

For example, if our observations are individuals, the rows might represent age, income, height, or years of education. Each individual is therefore represented by a point in an m-dimensional Euclidean space.

When $m = 2$, the observations form a collection of points in the plane. When $m = 3$, they form a cloud of points in three-dimensional space.

For larger m, direct visualization becomes difficult, but the underlying geometry remains unchanged.

Now imagine that this cloud of points is not distributed uniformly in every direction. Instead, it may be elongated along some particular direction.

Intuitively, that direction should contain more information about how the observations differ from one another than a direction along which the points are tightly clustered.

This observation provides the initial motivation for PCA.

The idea is to identify a new system of orthogonal coordinate axes that reflects the dominant directions of variation in the dataset.

Rather than describing each observation using all the original features, we may then describe it using only a few coordinates along these new axes.

But before proceeding, we must clarify what we actually mean by the variation of a dataset.

4.2 What Does Variance Mean Geometrically?

In elementary statistics, variance measures how widely observations are distributed around their mean.

For a collection of scalar observations $t_1, t_2, \ldots, t_n$, the sample variance is defined by

$$s^2 = \frac{1}{n-1} \sum_{j=1}^{n} (t_j - \bar{t})^2,$$

where

$$\bar{t} = \frac{1}{n} \sum_{j=1}^{n} t_j.$$

A large variance means that the observations are widely spread around their mean, whereas a small variance means that they are concentrated relatively close to it.

This interpretation is straightforward when the observations are real numbers.

But what happens when each observation is a vector in $\mathbb{R}^m$?

In that case, we must first specify the direction along which the spread is to be measured.

To understand this, imagine a two-dimensional dataset containing information about the income and expenditure of several individuals.

Suppose individuals with higher income also tend to have higher expenditure.

The corresponding data points may then form an elongated cloud approximately along a diagonal direction.

If we project these observations onto that diagonal line, their projected coordinates may be widely spread.

On the other hand, if we project the same observations onto a direction perpendicular to that line, their projected coordinates may lie much closer together.

Thus, the variance of the projected observations depends on the direction along which we examine the data.

This gives us a useful geometric interpretation:

Variance along a direction measures how widely the observations are spread when viewed from that particular direction.

Let $u \in \mathbb{R}^m$ be an arbitrary unit vector, so that

$$u^T u = 1.$$

For each observation x_j, consider its scalar projection onto u:

$$t_j = u^T x_j.$$

The mean of these projected coordinates is

$$\bar{t} = u^T \bar{x},$$

where

$$\bar{x} = \frac{1}{n} \sum_{j=1}^{n} x_j.$$

Therefore, the sample variance of the observations along the direction u is

$$\boxed{s_u^2 = \frac{1}{n-1} \sum_{j=1}^{n} \left(u^T (x_j - \bar{x})\right)^2.}$$

Notice something important.

Although the original dataset may contain hundreds of features, we can measure its spread along any chosen direction by first projecting the observations onto that direction and then computing the ordinary variance of the resulting scalar coordinates.

This leads us naturally to a mathematical question:

Among all possible unit directions, which one produces the greatest variance?

PCA is designed to answer precisely this question.

The first principal direction captures the maximum possible variance. Subsequent principal directions capture the largest remaining variance while being orthogonal to the previously selected directions.

This is the essential geometric principle behind PCA.

Of course, a direction with large variance is not necessarily important for every conceivable application. PCA uses variance as a measure of the dominant structure of a dataset.

The mathematical problem is therefore not to preserve every possible notion of information, but to preserve as much *variance* as possible using a prescribed number of orthogonal directions.

4.3 The Covariance Matrix

Having introduced the notion of variance along a particular direction, we now recall another important concept from elementary statistics: the covariance matrix.

While variance measures the spread of a single variable, covariance measures how two variables vary together.

For two variables X_i and X_j, with observations $x_{i\ell}$ and $x_{j\ell}$, their sample covariance is defined by

$$\mathrm{Cov}(X_i, X_j) = \frac{1}{n-1} \sum_{\ell=1}^{n} (x_{i\ell} - \bar{x}_i)(x_{j\ell} - \bar{x}_j).$$

Notice that when $i = j$, covariance reduces to the ordinary sample variance:

$$\mathrm{Cov}(X_i, X_i) = \mathrm{Var}(X_i).$$

The covariance matrix collects all these quantities into a single symmetric matrix:

$$C = \begin{bmatrix} \mathrm{Var}(X_1) & \cdots & \mathrm{Cov}(X_1, X_m) \\ \vdots & \ddots & \vdots \\ \mathrm{Cov}(X_m, X_1) & \cdots & \mathrm{Var}(X_m) \end{bmatrix}.$$

Thus, the diagonal entries represent the variances of individual features, while the off-diagonal entries describe how different features vary together.

Let us now see how this matrix arises naturally from matrix multiplication.

A Simple 2×2 Example

Suppose we have two features and two observations, represented by the matrix

$$X = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix}.$$

Following our convention, the columns represent observations and the rows represent features. Thus, x_{ij} denotes the value of the ith feature in the jth observation.

The mean of each feature is

$$\bar{x}_1 = \frac{x_{11} + x_{12}}{2}, \qquad \bar{x}_2 = \frac{x_{21} + x_{22}}{2}.$$

Subtracting the mean from each corresponding row, we obtain the centered data matrix

$$A = \begin{bmatrix} x_{11} - \bar{x}_1 & x_{12} - \bar{x}_1 \\ x_{21} - \bar{x}_2 & x_{22} - \bar{x}_2 \end{bmatrix}.$$

For convenience, let us introduce the notation

$$a_{ij} = x_{ij} - \bar{x}_i.$$

Then

$$\boxed{A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.}$$

We will now calculate the covariance matrix directly from the statistical definitions, without using matrix multiplication.

Computing the Covariance Matrix

Recall that the covariance matrix for two features has the form

$$C = \begin{bmatrix} \mathrm{Var}(X_1) & \mathrm{Cov}(X_1, X_2) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Var}(X_2) \end{bmatrix}.$$

Let us calculate each entry separately.

First diagonal entry:

The sample variance of the first feature is

$$\begin{aligned} C_{11} &= \mathrm{Var}(X_1) \\ &= \frac{1}{2-1}\left[(x_{11} - \bar{x}_1)^2 + (x_{12} - \bar{x}_1)^2\right] \\ &= a_{11}^2 + a_{12}^2. \end{aligned}$$

Second diagonal entry:

Similarly, the sample variance of the second feature is

$$
\begin{aligned}
C_{22} &= \mathrm{Var}(X_2) \\
&= \frac{1}{2-1}\left[(x_{21}-\bar{x}_2)^2 + (x_{22}-\bar{x}_2)^2\right] \\
&= a_{21}^2 + a_{22}^2.
\end{aligned}
$$

First off-diagonal entry:

The sample covariance between the first and second features is

$$
\begin{aligned}
C_{12} &= \mathrm{Cov}(X_1, X_2) \\
&= \frac{1}{2-1}\Big[(x_{11}-\bar{x}_1)(x_{21}-\bar{x}_2) \\
&\qquad\qquad + (x_{12}-\bar{x}_1)(x_{22}-\bar{x}_2)\Big] \\
&= a_{11}a_{21} + a_{12}a_{22}.
\end{aligned}
$$

Second off-diagonal entry:

By the symmetry of covariance,

$$
\begin{aligned}
C_{21} &= \mathrm{Cov}(X_2, X_1) \\
&= a_{21}a_{11} + a_{22}a_{12} \\
&= C_{12}.
\end{aligned}
$$

Putting these four entries together, we obtain the covariance matrix

$$
\boxed{C = \begin{bmatrix} a_{11}^2 + a_{12}^2 & a_{11}a_{21} + a_{12}a_{22} \\ a_{21}a_{11} + a_{22}a_{12} & a_{21}^2 + a_{22}^2 \end{bmatrix}.}
$$

So far, we have used nothing more than the elementary statistical definitions of variance and covariance.

Now comes an interesting observation.

The Same Calculation Through Matrix Multiplication

Recall that the centered data matrix is

$$
A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.
$$

Consider the matrix product

$$AA^T = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{bmatrix}.$$

Performing the multiplication, we obtain

$$\boxed{AA^T = \begin{bmatrix} a_{11}^2 + a_{12}^2 & a_{11}a_{21} + a_{12}a_{22} \\ a_{21}a_{11} + a_{22}a_{12} & a_{21}^2 + a_{22}^2 \end{bmatrix}.}$$

Compare this matrix with the covariance matrix we calculated earlier.

They are exactly the same!

In this particular example, the number of observations is $n = 2$, so the sample covariance normalization factor is

$$\frac{1}{n-1} = 1.$$

Hence,

$$C = AA^T.$$

But this equality without a normalization factor is specific to our example.

For a general dataset containing n observations, the sample covariance formula requires division by $n - 1$.

Thus, for a centered data matrix $A \in \mathbb{R}^{m \times n}$, with $n \geq 2$, we obtain

$$\boxed{C = \frac{1}{n-1} AA^T.}$$

To see this directly, the (i, j) entry of AA^T is

$$(AA^T)_{ij} = \sum_{\ell=1}^{n} a_{i\ell} a_{j\ell}.$$

Therefore,

$$\begin{aligned} C_{ij} &= \frac{1}{n-1} (AA^T)_{ij} \\ &= \frac{1}{n-1} \sum_{\ell=1}^{n} (x_{i\ell} - \bar{x}_i)(x_{j\ell} - \bar{x}_j). \end{aligned}$$

This is precisely the definition of sample covariance between the ith and jth features.

We have therefore established the covariance matrix formula directly from its elementary statistical definition.

Connection with PCA

The matrix AA^T, which we encountered earlier in our discussion of Singular Value Decomposition, is therefore closely connected to the statistical variation of the dataset.

More precisely, it is the covariance matrix multiplied by $n - 1$.

This connection is especially important for PCA.

Let $u \in \mathbb{R}^m$ be a unit vector, and let the centered data matrix be

$$A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}.$$

The scalar projection of the jth observation onto the direction u is

$$z_j = u^T a_j.$$

Since the observations have been centered, the mean of the projected coordinates is

$$\begin{aligned} \bar{z} &= \frac{1}{n} \sum_{j=1}^{n} z_j \\ &= \frac{1}{n} \sum_{j=1}^{n} u^T a_j \\ &= u^T \left(\frac{1}{n} \sum_{j=1}^{n} a_j \right) \\ &= 0. \end{aligned}$$

Therefore, by the definition of sample variance,

$$
\begin{aligned}
\mathrm{Var}(u^T A) &= \frac{1}{n-1}\sum_{j=1}^{n}(z_j - \bar{z})^2 \\
&= \frac{1}{n-1}\sum_{j=1}^{n}(u^T a_j)^2 \\
&= \frac{1}{n-1}\sum_{j=1}^{n}(u^T a_j)(a_j^T u) \\
&= \frac{1}{n-1}\sum_{j=1}^{n} u^T (a_j a_j^T) u \\
&= \frac{1}{n-1} u^T \left(\sum_{j=1}^{n} a_j a_j^T\right) u \\
&= \frac{1}{n-1} u^T A A^T u \\
&= u^T C u,
\end{aligned}
$$

where we have used the identity

$$
AA^T = \sum_{j=1}^{n} a_j a_j^T
$$

and the covariance matrix formula

$$
C = \frac{1}{n-1} AA^T.
$$

Hence, we have established

$$
\boxed{\mathrm{Var}(u^T A) = u^T C u = \frac{1}{n-1} u^T A A^T u.}
$$

This is a fundamental relationship in Principal Component Analysis.

It tells us that the variance of the observations along an arbitrary unit direction u can be expressed as a quadratic form involving the covariance matrix.

Consequently, finding the direction of maximum variance is equivalent to solving the optimization problem

$$
\boxed{\max_{u^T u = 1} u^T C u.}
$$

As we shall establish shortly, the solution is an eigenvector of C corresponding to its largest eigenvalue.

Thus, we can measure how much the data varies along any particular direction using a quadratic form involving the covariance matrix.

As we shall see, this leads naturally to the eigenvectors of the covariance matrix, or equivalently, the left singular vectors of the centered data matrix.

This is precisely where the statistical concept of variance meets the linear algebra of Singular Value Decomposition.

4.4 The Principal Component Analysis Procedure

As we have developed the necessary mathematical tools, the basic PCA procedure can now be summarized in a few relatively simple steps.

Although different textbooks may arrange the data matrix differently, we will consistently assume that features occupy the rows and observations occupy the columns.

Step 1: Center the Data

We begin by computing the mean observation vector

$$\bar{x} = \frac{1}{n} \sum_{j=1}^{n} x_j.$$

Each observation is then centered by subtracting this mean:

$$a_j = x_j - \bar{x}.$$

The resulting centered data matrix is

$$\boxed{A = X - \bar{x}\mathbf{1}_n^T,}$$

where $\mathbf{1}_n$ is the vector of n ones.

Notice that

$$\sum_{j=1}^{n} a_j = 0.$$

Consequently, the center of the data cloud has been translated to the origin.

Why do we perform this operation?

Recall that variance measures the spread of observations *around their mean.*

If we wish to identify directions of maximum variance, we should therefore examine the deviations from the mean rather than the absolute positions of the observations.

Mean centering allows us to measure this variation directly.

Indeed, for any unit vector u, the variance along u becomes

$$s_u^2 = \frac{1}{n-1}\sum_{j=1}^{n}(u^T a_j)^2.$$

We have already observed that this expression has a particularly convenient matrix representation.

From now on, unless stated otherwise, A denotes the centered data matrix.

Step 2: Compute the SVD

We compute the Singular Value Decomposition of A:

$$\boxed{A = U\Sigma V^T.}$$

Recall from Section 3 that the columns of U are the left singular vectors, while the columns of V are the right singular vectors.

The positive singular values are arranged in decreasing order:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0,$$

where $r = \text{rank}(A)$.

Since A is an $m \times n$ matrix, its left singular vectors belong to $\mathbb{R}^m$, the original feature space.

Thus, these vectors provide candidate directions along which we may describe the observations.

But why should SVD help us identify directions of maximum variance?

To appreciate this connection, recall the sample covariance matrix of the centered data:

$$\boxed{C = \frac{1}{n-1}AA^T,}$$

where we assume $n \geq 2$.

The covariance matrix describes how the original features vary together.

Its diagonal entries are the sample variances of the individual features, while the off-diagonal entries measure their pairwise covariances.

More importantly, for any unit direction u,

$$\boxed{s_u^2 = u^T C u.}$$

We have already derived this identity explicitly in the previous subsection.

Now recall from SVD that

$$AA^T = U\Sigma\Sigma^T U^T.$$

Consequently,

$$C = \frac{1}{n-1} U\Sigma\Sigma^T U^T.$$

Thus, the left singular vectors of A are eigenvectors of the covariance matrix C, with corresponding eigenvalues

$$\boxed{\lambda_i = \frac{\sigma_i^2}{n-1}.}$$

This is the first important connection between SVD and variance.

The squared singular values, after division by $n-1$, are precisely the variances associated with the corresponding principal directions.

Therefore, larger singular values correspond to directions along which the projected observations exhibit greater variance.

We will establish this conclusion rigorously below.

Step 3: Select the Principal Directions

Suppose we wish to represent the original m-dimensional observations using only k coordinates, where

$$1 \leq k \leq r.$$

We retain the first k columns of U:

$$\boxed{U_k = \begin{bmatrix} u_1 & u_2 & \cdots & u_k \end{bmatrix} \in \mathbb{R}^{m \times k}.}$$

These vectors form an orthonormal basis for a k-dimensional subspace of the original feature space.

We call this the *principal subspace*.

Why do we retain the first k vectors rather than some other k vectors?

Because the singular values are arranged in decreasing order, the selected vectors correspond to the largest variances

$$\frac{\sigma_1^2}{n-1}, \ldots, \frac{\sigma_k^2}{n-1}.$$

Thus, PCA retains those orthogonal directions along which the observations exhibit the greatest total variation.

The total sample variance across all original features is

$$\begin{aligned}
\operatorname{tr}(C) &= \operatorname{tr}\left(\frac{1}{n-1} AA^T\right) \\
&= \frac{1}{n-1} \operatorname{tr}(AA^T) \\
&= \frac{1}{n-1} \operatorname{tr}\left(U\Sigma V^T V \Sigma^T U^T\right) \\
&= \frac{1}{n-1} \operatorname{tr}\left(U\Sigma\Sigma^T U^T\right) \\
&= \frac{1}{n-1} \operatorname{tr}\left(\Sigma\Sigma^T U^T U\right) \\
&= \frac{1}{n-1} \operatorname{tr}\left(\Sigma\Sigma^T\right) \\
&= \frac{1}{n-1} \sum_{i=1}^{r} \sigma_i^2 \\
&= \frac{1}{n-1} \|A\|_F^2.
\end{aligned}$$

Here we have used the orthogonality of U and V, together with the cyclic property of the trace,

$$\operatorname{tr}(BC) = \operatorname{tr}(CB),$$

whenever both products are defined.

We have also used the identity

$$\|A\|_F^2 = \operatorname{tr}(AA^T) = \sum_{i=1}^{r} \sigma_i^2.$$

Consequently,

$$\boxed{\operatorname{tr}(C) = \frac{1}{n-1}\|A\|_F^2 = \frac{1}{n-1}\sum_{i=1}^{r}\sigma_i^2.}$$

For a nonzero centered dataset, the proportion of this variance retained by the first k principal directions is

$$\boxed{\mathrm{EVR}_k = \frac{\sum_{i=1}^{k}\sigma_i^2}{\sum_{i=1}^{r}\sigma_i^2}.}$$

This quantity is called the *cumulative explained variance ratio*.

It allows us to measure how much of the original variation is retained after dimensionality reduction.

Notice that U_k contains directions, not transformed observations.

To obtain the new coordinates of our observations, one additional operation is required.

Step 4: Project the Data onto the Principal Directions

We compute

$$\boxed{Z = U_k^T A.}$$

Let us examine the dimensions:

$$\underbrace{U_k^T}_{k\times m}\underbrace{A}_{m\times n} = \underbrace{Z}_{k\times n}.$$

The jth column of Z is

$$z_j = U_k^T a_j.$$

In particular,

$$z_j = \begin{bmatrix} u_1^T a_j \\ u_2^T a_j \\ \vdots \\ u_k^T a_j \end{bmatrix}.$$

Thus, each entry measures how much the observation extends along a particular principal direction.

The original observation $a_j \in \mathbb{R}^m$ has now been represented by a new vector

$$z_j \in \mathbb{R}^k.$$

This is the actual dimensionality reduction.

The matrix Z is commonly called the matrix of *principal component scores*.

An interesting observation follows directly from SVD. Since

$$A = U\Sigma V^T,$$

and the columns of U are orthonormal,

$$\boxed{Z = U_k^T A = \Sigma_k V_k^T,}$$

where Σ_k is the $k \times k$ diagonal matrix of the retained singular values, and V_k contains the corresponding right singular vectors.

Therefore, the principal component scores are precisely the corresponding right singular-vector coordinates scaled by the singular values.

Now let us return once more to the concept of variance.

The ith row of Z contains the projected coordinates of all observations along the direction u_i.

Since the data is centered, the sample variance of these coordinates is

$$\mathrm{Var}(\mathrm{PC}_i) = \frac{1}{n-1}\|u_i^T A\|_2^2.$$

But

$$u_i^T A = \sigma_i v_i^T.$$

Since $\|v_i\|_2 = 1$,

$$\boxed{\mathrm{Var}(\mathrm{PC}_i) = \frac{\sigma_i^2}{n-1}.}$$

Thus, the singular values tell us exactly how much variance is captured by each principal component.

There is another important property of the transformed data.

Since

$$Z = \Sigma_k V_k^T,$$

we obtain

$$\frac{1}{n-1} Z Z^T = \frac{1}{n-1} \Sigma_k^2.$$

Hence, the covariance matrix of the principal component scores is diagonal:

$$\boxed{\mathrm{Cov}(Z) = \frac{1}{n-1} \begin{bmatrix} \sigma_1^2 & & 0 \\ & \ddots & \\ 0 & & \sigma_k^2 \end{bmatrix}.}$$

Consequently, the principal components are mutually uncorrelated.

This gives another interpretation of PCA: it constructs new uncorrelated coordinates, ordered according to the amount of variance they capture.

Step 5: Reconstruct the Original Observations

Suppose we now wish to return from the reduced k-dimensional representation to the original feature space.

We compute

$$\boxed{\widehat{A} = U_k Z.}$$

Substituting $Z = U_k^T A$,

$$\boxed{\widehat{A} = U_k U_k^T A.}$$

Notice the dimensions:

$$\underbrace{U_k}_{m \times k} \underbrace{Z}_{k \times n} = \underbrace{\widehat{A}}_{m \times n}.$$

Thus, the reconstructed data matrix has the same dimensions as the original centered data matrix.

However, in general,

$$\widehat{A} \neq A.$$

The reconstructed observations retain only the components lying in the selected principal subspace.

The components orthogonal to that subspace are discarded.

If we wish to reconstruct the observations on their original, uncentered scale, we must also restore the mean:

$$\boxed{\widehat{X} = \widehat{A} + \bar{x}\mathbf{1}_n^T.}$$

This completes the basic PCA procedure.

4.5 What Have We Actually Accomplished?

It is worth reflecting on what these operations have achieved.

Originally, every observation was represented by m features having familiar interpretations, such as age, income, and education.

After the transformation

$$Z = U_k^T A,$$

the observations are described using only k coordinates.

However, the new coordinates no longer represent individual features in the original sense.

For example, a principal component might be a linear combination of age, income, and education, rather than any one of these quantities individually.

Thus, dimensionality reduction comes with a trade-off: the new coordinates may be mathematically more economical, but their interpretation is often less immediate.

Nevertheless, the transformation is not mysterious.

The coefficients defining each new coordinate are stored explicitly in the columns of U_k.

When we reconstruct the observations through

$$\widehat{A} = U_k Z,$$

we return to the original feature coordinates.

The rows of $\widehat{A}$ can therefore again be interpreted as age, income, education, and so forth.

Their numerical values, however, are generally different from those in the original matrix.

Geometrically, each observation has been replaced by its orthogonal projection onto the principal subspace.

This process may suppress small-scale fluctuations and can be useful for denoising when the discarded directions predominantly contain noise.

However, PCA does not automatically distinguish noise from meaningful variation. Information contained in discarded directions is lost, whether it represents noise or an important feature of the dataset.

We should also remember that variance depends on the scale of the features.

For example, features measured in dollars may numerically dominate features measured in years. Therefore, depending on the application, it may be appropriate to standardize the features before applying PCA.

Such standardization changes the mathematical problem, since PCA then operates on the scaled data rather than the original measurements.

4.6 The Connection with Reconstruction Error

There is another way to understand the same result.

Suppose we choose any unit vector u and reconstruct each observation from its scalar projection onto that direction.

For the jth observation,

$$\widehat{a}_j = u(u^T a_j).$$

The reconstruction error is

$$e_j = a_j - \widehat{a}_j.$$

Because this is an orthogonal projection, the error vector is perpendicular to the reconstructed vector.

The Pythagorean theorem therefore gives

$$\|a_j\|_2^2 = \|\widehat{a}_j\|_2^2 + \|a_j - \widehat{a}_j\|_2^2.$$

Since $\|u\|_2 = 1$,

$$\|\widehat{a}_j\|_2^2 = (u^T a_j)^2.$$

It follows that

$$\|a_j - \widehat{a}_j\|_2^2 = \|a_j\|_2^2 - (u^T a_j)^2.$$

Summing over all observations and using the Frobenius norm,

$$\boxed{\|A - uu^T A\|_F^2 = \|A\|_F^2 - u^T AA^T u.}$$

Here,

$$\|A\|_F^2 = \sum_{i,j} A_{ij}^2.$$

The quantity $\|A\|_F^2$ does not depend on u.

Consequently, minimizing the reconstruction error is equivalent to maximizing the retained variance.

Thus,

$$\boxed{\begin{array}{c} \max\limits_{\|u\|_2=1} u^T AA^T u \\ \\ \Longleftrightarrow \\ \\ \min\limits_{\|u\|_2=1} \|A - uu^T A\|_F^2. \end{array}}$$

This equivalence reveals the geometric beauty underlying PCA.

The direction that captures the greatest variance is also the direction that produces the smallest squared reconstruction error among all one-dimensional orthogonal projections.

More generally, if $Q \in \mathbb{R}^{m \times k}$ has orthonormal columns, then

$$\boxed{\|A - QQ^T A\|_F^2 = \|A\|_F^2 - \|Q^T A\|_F^2.}$$

Thus, maximizing the total variation retained in the k-dimensional subspace is equivalent to minimizing its squared reconstruction error.

For PCA, we choose $Q = U_k$.

4.7 The Optimality Question

We have obtained the PCA reconstruction

$$\widehat{A} = U_k U_k^T A.$$

Using the singular value decomposition, this can also be written as

$$\boxed{\widehat{A} = \sum_{i=1}^{k} \sigma_i u_i v_i^T.}$$

Since the singular vectors are orthonormal, the discarded part is

$$A - \widehat{A} = \sum_{i=k+1}^{r} \sigma_i u_i v_i^T.$$

Consequently, its squared Frobenius norm is

$$\boxed{\|A - \widehat{A}\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2.}$$

Similarly, when $k < r$, the spectral norm of the residual is

$$\boxed{\|A - \widehat{A}\|_2 = \sigma_{k+1}.}$$

These formulas tell us exactly how much error is introduced by discarding the smaller singular values.

But an important question still remains.

We have shown that PCA selects an optimal k-dimensional orthogonal projection under the squared Frobenius reconstruction error.

What if we do not restrict ourselves to orthogonal projections?

Suppose we allow any matrix B of rank at most k to approximate A.

Could some other construction produce a smaller error?

Remarkably, the answer is no.

The Eckart–Young theorem establishes that the truncated SVD minimizes the approximation error among all matrices of rank at most k, both in the Frobenius and spectral norms.

In particular,

$$\boxed{\min_{\operatorname{rank}(B) \le k} \|A - B\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2,}$$

and, for $k < r$,

$$\boxed{\min_{\operatorname{rank}(B) \le k} \|A - B\|_2 = \sigma_{k+1}.}$$

These are substantially stronger claims than the elementary projection argument alone establishes.

We shall therefore devote the later sections of this note to developing and proving these results rigorously.

For now, the essential observation is that PCA is not simply a procedure for discarding coordinates.

It is a geometrically motivated optimization method whose optimality can be characterized precisely through the singular values of the data matrix.

This is the mathematical connection that we seek to understand in the remainder of this note.

5 Mathematical Justification of Optimal Low-Rank Approximation: The Eckart–Young Theorem

In the preceding section, we saw that PCA selects a low-dimensional subspace and reconstructs the centered data through an orthogonal projection. We also found that the squared reconstruction error is determined by the singular values that have been discarded.

However, there is an important distinction between calculating the error of a particular approximation and proving that no other approximation of same rank can do better.

So far, we have focused on approximations arising from orthogonal projections. What if we consider every possible matrix of rank at most k, without requiring it to arise from a projection?

The Eckart–Young theorem answers this stronger question. We will first state the result and then prove it for two important matrix norms.

5.1 The Approximation Problem and Matrix Norms

Let $A \in \mathbb{R}^{m\times n}$ be any real matrix of rank r, with singular value decomposition

$$A = \sum_{i=1}^{r} \sigma_i u_i v_i^T, \qquad \sigma_1 \geq \cdots \geq \sigma_r > 0.$$

For $0 \leq k < r$, define the truncated singular value decomposition by

$$\boxed{A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T,}$$

where the empty sum for $k = 0$ is understood to be the zero matrix. Because the singular vectors are orthonormal, A_k has rank at most k (in fact, rank k when $k > 0$).

Our question is whether there exists some other matrix B satisfying

$$\operatorname{rank}(B) \le k$$

that approximates A more accurately. Of course, to make the word "accurately" mathematically meaningful, we must first specify how approximation error is measured.

The Frobenius norm.

For $M = (m_{ij})$, define

$$\boxed{\|M\|_F = \left(\sum_{i,j} m_{ij}^2 \right)^{1/2}.}$$

This measures the total entrywise error. In particular, $\|A-B\|_F^2$ is the sum of the squared differences between all corresponding entries of A and B.

The spectral norm.

The matrix spectral norm, also called the induced Euclidean operator norm, is

$$\boxed{\|M\|_2 = \max_{\|x\|_2=1} \|Mx\|_2.}$$

It measures the greatest length that M can produce when acting on a unit vector. For an approximation error $M = A - B$, it measures the largest possible directional error.

Thus, these norms ask somewhat different questions: the Frobenius norm measures the total squared error across the matrix, whereas the spectral norm measures the worst error over all unit input directions.

5.2 Statement of the Eckart–Young Theorem

Theorem 5.1 (Eckart–Young–Mirsky)**.** *Let $A \in \mathbb{R}^{m\times n}$ have rank r and positive singular values $\sigma_1 \ge \cdots \ge \sigma_r > 0$. For an integer $0 \le k < r$, define*

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T.$$

Then, among all matrices B of rank at most k, the truncated SVD A_k minimizes both the Frobenius norm and the spectral norm of the error. More precisely,

$$\boxed{\min_{\operatorname{rank}(B)\le k} \|A - B\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2,}$$

and

$$\boxed{\min_{\operatorname{rank}(B)\le k} \|A - B\|_2 = \sigma_{k+1}.}$$

Both minima are attained by $B = A_k$.

The key phrase is *among all matrices of rank at most* k.

The theorem does not assume that B is formed by projection, nor does it require that B share the singular vectors of A.

To establish the theorem, we must show two things for each norm: every admissible B has error at least as large as the claimed bound, and the truncated SVD actually attains that bound.

5.3 Proof Under the Frobenius Norm

Proof. The case $k = 0$ is immediate, since the only matrix of rank zero is $B = 0$. We therefore assume $1 \le k < r$.

Step 1: Begin with an arbitrary rank-k approximation.

Let $B \in \mathbb{R}^{m\times n}$ be *any* matrix with

$$\operatorname{rank}(B) \le k.$$

Its column space has dimension at most k. Since $k < m$, we can choose an orthonormal matrix

$$Q \in \mathbb{R}^{m\times k}, \qquad Q^T Q = I_k,$$

such that the column space of B is contained in $\operatorname{Col}(Q)$.

Set

$$P = QQ^T.$$

Then P is the orthogonal projector onto $\operatorname{Col}(Q)$, and because every column of B belongs to this subspace,

$$\boxed{PB = B.}$$

Step 2: Separate the approximation error into orthogonal parts.

Write

$$\begin{aligned} A - B &= (A - PA) + (PA - B) \\ &= (I - P)A + (PA - B). \end{aligned}$$

Every column of $(I - P)A$ is orthogonal to $\mathrm{Col}(Q)$, whereas every column of $PA - B$ lies inside that subspace.

Consequently, these two matrices are orthogonal in the Frobenius inner product, and the Pythagorean theorem gives

$$\boxed{\|A - B\|_F^2 = \|(I - P)A\|_F^2 + \|PA - B\|_F^2.}$$

In particular,

$$\boxed{\|A - B\|_F^2 \geq \|(I - P)A\|_F^2.}$$

Thus, no matrix B whose columns lie in this subspace can approximate A more accurately than the orthogonal projection PA.

Step 3: Bound the variance that an arbitrary k-dimensional subspace can retain.

Using the same orthogonal decomposition,

$$\begin{aligned} \|(I - P)A\|_F^2 &= \|A\|_F^2 - \|PA\|_F^2 \\ &= \|A\|_F^2 - \|Q^T A\|_F^2. \end{aligned}$$

We now show that

$$\boxed{\|Q^T A\|_F^2 \leq \sum_{i=1}^{k} \sigma_i^2.}$$

Recall from SVD that

$$AA^T = \sum_{i=1}^{r} \sigma_i^2 u_i u_i^T.$$

Therefore,

$$\begin{aligned} \|Q^T A\|_F^2 &= \mathrm{tr}(Q^T AA^T Q) \\ &= \sum_{i=1}^{r} \sigma_i^2 \,\mathrm{tr}(Q^T u_i u_i^T Q) \\ &= \sum_{i=1}^{r} \sigma_i^2 \|Q^T u_i\|_2^2. \end{aligned}$$

Put $p_i = \|Q^T u_i\|_2^2$ and extend the singular values by $\sigma_i = 0$ for $r < i \leq m$. Since Q has orthonormal columns,

$$0 \leq p_i \leq 1, \qquad \sum_{i=1}^{m} p_i = k.$$

The last identity follows from

$$\begin{aligned}\sum_{i=1}^{m} p_i &= \operatorname{tr}\left(Q^T \left(\sum_{i=1}^{m} u_i u_i^T\right) Q\right) \\ &= \operatorname{tr}(Q^T Q) = k.\end{aligned}$$

We now wish to establish the inequality

$$\sum_{i=1}^{m} \sigma_i^2 p_i \leq \sum_{i=1}^{k} \sigma_i^2.$$

Let us divide the sum into two parts:

$$\sum_{i=1}^{m} \sigma_i^2 p_i = \sum_{i=1}^{k} \sigma_i^2 p_i + \sum_{i=k+1}^{m} \sigma_i^2 p_i.$$

Since $\sigma_i^2 \leq \sigma_k^2$ whenever $i > k$, and $p_i \geq 0$, we have

$$\sum_{i=k+1}^{m} \sigma_i^2 p_i \leq \sigma_k^2 \sum_{i=k+1}^{m} p_i.$$

Consequently,

$$\sum_{i=1}^{m} \sigma_i^2 p_i \leq \sum_{i=1}^{k} \sigma_i^2 p_i + \sigma_k^2 \sum_{i=k+1}^{m} p_i.$$

Now recall that

$$\sum_{i=1}^{m} p_i = k.$$

Therefore,

$$\begin{aligned}\sum_{i=k+1}^{m} p_i &= k - \sum_{i=1}^{k} p_i \\ &= \sum_{i=1}^{k} (1 - p_i).\end{aligned}$$

Substituting this expression into the previous inequality gives

$$\begin{aligned}\sum_{i=1}^{m} \sigma_i^2 p_i &\leq \sum_{i=1}^{k} \sigma_i^2 p_i + \sigma_k^2 \sum_{i=1}^{k} (1 - p_i) \\ &= \sum_{i=1}^{k} \left[\sigma_i^2 p_i + \sigma_k^2 (1 - p_i)\right].\end{aligned}$$

Now here comes the crucial observation.

For every $1 \leq i \leq k$,

$$\sigma_i^2 \geq \sigma_k^2.$$

Also, since $0 \leq p_i \leq 1$,

$$1 - p_i \geq 0.$$

Hence,

$$\sigma_k^2 (1 - p_i) \leq \sigma_i^2 (1 - p_i).$$

Adding $\sigma_i^2 p_i$ to both sides, we obtain

$$\begin{aligned}\sigma_i^2 p_i + \sigma_k^2 (1 - p_i) &\leq \sigma_i^2 p_i + \sigma_i^2 (1 - p_i) \\ &= \sigma_i^2 [p_i + (1 - p_i)] \\ &= \sigma_i^2.\end{aligned}$$

Summing this inequality from $i = 1$ to k, we finally obtain

$$\boxed{\sum_{i=1}^{m} \sigma_i^2 p_i \leq \sum_{i=1}^{k} \sigma_i^2.}$$

Equality is attained by choosing $Q = U_k$, for which

$$p_i = \begin{cases} 1, & 1 \le i \le k, \\ 0, & i > k. \end{cases}$$

Thus, the first k left singular vectors maximize the quantity

$$\|Q^T A\|_F^2 = \sum_{i=1}^{m} \sigma_i^2 p_i$$

over all matrices Q with orthonormal columns.

This proves the claimed bound on $\|Q^T A\|_F^2$.

Step 4: Obtain a lower bound for every admissible B.

Combining our inequalities,

$$\begin{aligned} \|A - B\|_F^2 &\ge \|(I - P)A\|_F^2 \\ &= \|A\|_F^2 - \|Q^T A\|_F^2 \\ &\ge \sum_{i=1}^{r} \sigma_i^2 - \sum_{i=1}^{k} \sigma_i^2 \\ &= \sum_{i=k+1}^{r} \sigma_i^2. \end{aligned}$$

This inequality holds for *every* B satisfying $\operatorname{rank}(B) \le k$.

Step 5: Show that the truncated SVD achieves the bound.

For

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T,$$

we have

$$A - A_k = \sum_{i=k+1}^{r} \sigma_i u_i v_i^T.$$

The rank-one matrices $u_i v_i^T$ are orthonormal under the Frobenius inner product. Therefore,

$$\boxed{\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2.}$$

The lower bound is attained, proving

$$\boxed{\min_{\operatorname{rank}(B)\leq k} \|A - B\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2.}$$

□

Notice what made this proof work.

We did not begin by assuming that B had the same singular vectors as A. Instead, we allowed an arbitrary rank-k matrix and showed that an orthogonal projection onto its column space, or a larger k-dimensional subspace, could never have a greater error than B.

We then used SVD to bound how much variation any such subspace can retain.

5.4 Proof Under the Spectral Norm

The spectral-norm proof uses a different idea. Rather than summing errors over all entries, we need only identify *one* unit vector on which every rank-k approximation must incur an error of at least σ_{k+1}.

Proof. Let $B \in \mathbb{R}^{m\times n}$ be any matrix with $\operatorname{rank}(B) \leq k$.

Step 1: Choose a subspace of dimension $k+1$.

Consider the first $k+1$ right singular vectors of A and define

$$W = \operatorname{span}\{v_1, \ldots, v_{k+1}\}.$$

Because these vectors are orthonormal,

$$\dim(W) = k + 1.$$

Step 2: Find a unit vector that B sends to zero.

Restrict the transformation B to W:

$$B|_W : W \longrightarrow \mathbb{R}^m.$$

Its rank cannot exceed k. By the rank-nullity theorem,

$$\begin{aligned} \dim \ker(B|_W) &= \dim(W) - \operatorname{rank}(B|_W) \\ &\geq (k+1) - k = 1. \end{aligned}$$

Therefore, there exists a unit vector $x \in W$ such that

$$\boxed{Bx = 0, \qquad \|x\|_2 = 1.}$$

This is the crucial observation: any matrix of rank at most k necessarily loses at least one nonzero direction from the $(k+1)$-dimensional space W.

Step 3: Estimate the error on this special vector.

Since $x \in W$, write

$$x = \sum_{i=1}^{k+1} c_i v_i, \qquad \sum_{i=1}^{k+1} c_i^2 = 1.$$

Using $Av_i = \sigma_i u_i$, we obtain

$$Ax = \sum_{i=1}^{k+1} c_i \sigma_i u_i.$$

The left singular vectors are orthonormal, so

$$\begin{aligned} \|Ax\|_2^2 &= \sum_{i=1}^{k+1} c_i^2 \sigma_i^2 \\ &\geq \sigma_{k+1}^2 \sum_{i=1}^{k+1} c_i^2 \\ &= \sigma_{k+1}^2. \end{aligned}$$

But $Bx = 0$, and hence

$$\|(A-B)x\|_2 = \|Ax\|_2 \geq \sigma_{k+1}.$$

By the definition of the spectral norm,

$$\boxed{\|A-B\|_2 = \max_{\|y\|_2=1} \|(A-B)y\|_2 \geq \sigma_{k+1}.}$$

Thus, no rank-k matrix can achieve a smaller spectral-norm error.

Step 4: Show that A_k achieves the lower bound.

Recall that

$$A - A_k = \sum_{i=k+1}^{r} \sigma_i u_i v_i^T.$$

This residual has singular values $\sigma_{k+1}, \ldots, \sigma_r$. Its largest singular value is σ_{k+1}. Therefore,

$$\boxed{\|A - A_k\|_2 = \sigma_{k+1}.}$$

Combining the lower bound with this equality yields

$$\boxed{\min_{\text{rank}(B) \leq k} \|A - B\|_2 = \sigma_{k+1}.}$$

□

5.5 What Do These Results Tell Us About PCA?

Let us now return to the centered data matrix used in Principal Component Analysis.

Its truncated SVD satisfies

$$\begin{aligned} A_k &= \sum_{i=1}^{k} \sigma_i u_i v_i^T \\ &= U_k U_k^T A. \end{aligned}$$

Thus, the matrix obtained by projecting onto the principal subspace and reconstructing the observations is exactly the truncated SVD.

The Eckart–Young theorem now supplies the promised global optimality statement:

$$\boxed{\begin{aligned} \|A - A_k\|_F &\leq \|A - B\|_F, \\ \|A - A_k\|_2 &\leq \|A - B\|_2 \end{aligned}}$$

for every $B \in \mathbb{R}^{m \times n}$ with $\text{rank}(B) \leq k$.

The Frobenius result minimizes total squared reconstruction error, while the spectral result minimizes the worst directional reconstruction error.

The theorem does not require the competing approximation B to be an orthogonal projection.

There is also a subtle point about uniqueness. The theorem guarantees that the truncated SVD is optimal, but does not guarantee that it is the only optimal approximation.

In particular, repeated singular values may lead to different optimal choices, which we may later discuss in another note.

We have therefore connected the statistical objective of preserving variance to the linear-algebraic objective of optimal low-rank approximation.

What began as a geometric intuition is now supported by a rigorous theorem.

6 Numerical Implementation of PCA Using Python

Having established the mathematical foundations of Principal Component Analysis and proved the Eckart–Young theorem, we now illustrate these results through numerical experiments using Python.

We implement PCA directly using NumPy's Singular Value Decomposition rather than relying on a prebuilt PCA algorithm.

Throughout our implementation, we follow the convention adopted in this note:

$$X \in \mathbb{R}^{m \times n},$$

where rows represent features and columns represent observations.

6.1 Implementation Using a Small Dataset

We begin with a simple dataset containing two features and four observations:

$$X = \begin{bmatrix} 8 & 9 & 11 & 12 \\ 4 & 4 & 6 & 6 \end{bmatrix}.$$

We first center the data by subtracting the mean of each feature. We then compute the covariance matrix and the Singular Value Decomposition of the centered data.

Retaining only the first principal direction, we reduce the original two-dimensional observations to one-dimensional coordinates.

The numerical experiment demonstrates that the first principal component preserves approximately 97.92% of the total variance.

We also reconstruct the observations and verify that the squared Frobenius error and spectral error agree with the formulas established by the Eckart–Young theorem.

6.2 Implementation Using a Real Dataset

Next, we consider a larger dataset consisting of handwritten digit images.

The dataset contains 1797 observations, each represented by 64 numerical features corresponding to the pixels of an 8×8 image.

Following our convention, the data matrix has dimensions

$$X \in \mathbb{R}^{64\times 1797}.$$

After centering the observations, we compute the SVD and select the smallest number of principal components required to preserve at least 90% of the total variance.

In this experiment, 21 principal components preserve approximately 90.32% of the total variance.

Thus, we reduce the original 64 features to only 21 principal component coordinates.

We subsequently reconstruct the original images and compare them with their low-rank approximations.

Finally, we verify numerically that

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2$$

and

$$\|A - A_k\|_2 = \sigma_{k+1},$$

confirming the error identities established by the Eckart–Young theorem.

6.3 Python Source Code

The complete Python implementation for both experiments is provided below.

The program also produces graphical illustrations of the principal directions, explained variance, and reconstructed observations.

Listing 1: Numerical implementation of PCA and verification of the Eckart–Young theorem

```
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

# No forced 'Agg' backend: forcing it would HIDE figures in Jupyter.
np.set_printoptions(precision=5, suppress=True)
OUTPUT_DIR = (Path(__file__).resolve().parent
              if '__file__' in globals() else Path.cwd())


# ============================================================
# IMPLEMENTATION 1: SMALL, HAND-ENTERED DATASET
# ============================================================
def run_small_example():
    # ------------------------------------------------------------
    # Step 1. Enter observations as COLUMNS; features are ROWS.
    # ------------------------------------------------------------
    X = np.array([
```

```
        [8, 9, 11, 12],  # Feature 1, measured for four observations
        [4, 4,  6,  6],  # Feature 2, measured for four observations
    ], dtype=float)

    m, n = X.shape
    assert (m, n) == (2, 4)

    mean = X.mean(axis=1, keepdims=True)  # (m, 1): mean of each ROW
    A = X - mean                          # (m, n): centered data

    print("1. DATA AND CENTERING")
    print("X =\n", X)
    print("Feature means =\n", mean)
    print("A = X - mean =\n", A)
    print("Shape (features, observations):", A.shape)
    assert np.allclose(A.sum(axis=1), 0.0)

    # ------------------------------------------------------------------
    # Step 2. Build the sample covariance matrix: C = A A^T/(n-1).
    # ------------------------------------------------------------------
    C = (A @ A.T) / (n - 1)
    print("\n2. SAMPLE COVARIANCE")
    print("C = A @ A.T / (n - 1) =\n", C)
    print("Individual feature variances:", np.diag(C))
    print("Total sample variance = tr(C):", np.trace(C))
    assert np.allclose(C, np.cov(X, rowvar=True, ddof=1))

    # ------------------------------------------------------------------
    # Step 3. SVD of CENTERED data, not of the covariance matrix.
    # ------------------------------------------------------------------
    U, s, Vt = np.linalg.svd(A, full_matrices=False)
    pc_variances = s**2 / (n - 1)
    explained_ratios = s**2 / np.sum(s**2)

    print("\n3. SINGULAR VALUE DECOMPOSITION")
    print("U (principal directions) =\n", U)
    print("Singular values =", s)
    print("V.T =\n", Vt)
    print("Variance along principal directions =", pc_variances)
    print("Explained-variance ratios =", explained_ratios)
    assert np.allclose(A, U @ np.diag(s) @ Vt)
    assert np.allclose(U.T @ C @ U, np.diag(pc_variances))
    # Singular vector signs may differ from other computers; that is normal.

    # ------------------------------------------------------------------
    # Step 4. Keep k=1 principal component and reconstruct.
    # ------------------------------------------------------------------
    k = 1
    Uk = U[:, :k]       # (m, k): first k LEFT singular vectors
    Z = Uk.T @ A        # (k, n): one reduced coordinate per observation
    Ahat = Uk @ Z       # (m, n): centered rank-k reconstruction
    Xhat = Ahat + mean  # (m, n): restore the means of the features
    Ak = (U[:, :k] * s[:k]) @ Vt[:k, :]  # Same truncated SVD

    print("\n4. PROJECTION AND RECONSTRUCTION")
    print("U_k shape:", Uk.shape, " Z shape:", Z.shape)
    print("Principal direction U_k =\n", Uk)
    print("Scores Z = U_k.T @ A =\n", Z)
    print("Reconstructed centered data Ahat =\n", Ahat)
    print("Reconstructed raw data Xhat =\n", Xhat)
    assert np.allclose(Ahat, Ak)
    assert np.linalg.matrix_rank(Ahat) == k

    # ------------------------------------------------------------------
    # Step 5. Verify BOTH Eckart--Young error formulas.
    # ------------------------------------------------------------------
    E = A - Ahat
    fro_squared = np.linalg.norm(E, ord="fro")**2
```

```
    spectral_error = np.linalg.norm(E, ord=2)
    fro_prediction = np.sum(s[k:]**2)
    spectral_prediction = s[k]  # k < rank(A) in this example
    retained_variance = np.sum(s[:k]**2) / np.sum(s**2)

    print("\n5. ECKART--YOUNG VERIFICATION")
    print(f"Variance retained: {retained_variance:.4%}")
    print(f"Frobenius error squared:   {fro_squared:.8f}")
    print(f"Sum of discarded sigma^2: {fro_prediction:.8f}")
    print(f"Spectral error:           {spectral_error:.8f}")
    print(f"Next singular value:      {spectral_prediction:.8f}")
    print(f"Relative squared error:   {fro_squared / np.sum(s**2):.4%}")
    print(f"1 - explained variance:   {1.0 - retained_variance:.4%}")
    assert np.isclose(fro_squared, fro_prediction)
    assert np.isclose(spectral_error, spectral_prediction)
    assert np.isclose(fro_squared / np.sum(s**2), 1.0 - retained_variance)

    # Compare two other one-dimensional subspaces with PCA's optimal one.
    horizontal = np.array([[1.0], [0.0]])
    diagonal = np.array([[1.0], [1.0]]) / np.sqrt(2.0)
    print("\n6. OTHER ONE-DIMENSIONAL PROJECTIONS")
    for name, q in [("horizontal", horizontal),
                    ("45-degree diagonal", diagonal),
                    ("optimal PCA direction", Uk)]:
        B = q @ (q.T @ A)
        error_sq = np.linalg.norm(A - B, ord="fro")**2
        print(f"{name:>22}: squared Frobenius error = {error_sq:.6f}")
        assert error_sq + 1e-10 >= fro_squared

    # ------------------------------------------------------------
    # Step 6. Plot the points and their orthogonal projections.
    # ------------------------------------------------------------
    fig, ax = plt.subplots(figsize=(7, 6))
    ax.scatter(A[0], A[1], s=75, label="Original centered observations")
    ax.scatter(Ahat[0], Ahat[1], s=85, marker="x",
               label="Rank-1 reconstruction")
    for j in range(n):
        ax.plot([A[0, j], Ahat[0, j]], [A[1, j], Ahat[1, j]],
                linestyle="--", linewidth=1)
        ax.annotate(str(j + 1), (A[0, j], A[1, j]),
                    xytext=(5, 5), textcoords="offset points")
    t = np.linspace(-3, 3, 101)
    ax.plot(t * Uk[0, 0], t * Uk[1, 0], label="First principal axis")
    ax.axhline(0, linewidth=0.6)
    ax.axvline(0, linewidth=0.6)
    ax.set(xlabel="Centered feature 1", ylabel="Centered feature 2",
           title="Implementation 1: PCA with observations as columns")
    ax.set_aspect("equal", adjustable="box")
    ax.legend()
    ax.grid(alpha=0.25)
    fig.tight_layout()
    destination = OUTPUT_DIR / "implementation1_projection.png"
    fig.savefig(destination, dpi=250)
    plt.show()  # Display immediately in Jupyter / an interactive session.
    plt.close(fig)
    print("\nFigure saved:", destination)
    print("All Implementation 1 mathematical checks passed.")


# ============================================================
# IMPLEMENTATION 2: IMPORTED HANDWRITTEN-DIGITS DATASET
# ============================================================
def run_digits_example():
    # ------------------------------------------------------------
    # Step 1. Import a real dataset and TRANSPOSE to our convention.
    # ------------------------------------------------------------
    digits = load_digits()
    # sklearn provides digits.data as (observations, features) = (1797, 64).
```

```
    # We TRANSPOSE it: X is (64 features, 1797 observations).
    X = digits.data.T.astype(float)
    labels = digits.target  # One digit label for each COLUMN of X.
    m, n = X.shape
    assert m == 64 and n == len(labels)

    print("1. LOADING THE HANDWRITTEN-DIGITS DATASET")
    print("Shape of X (features, observations):", X.shape)
    print("Each column reshapes to an 8 x 8 handwritten-digit image.")
    print("The labels are used only for visualizing the results.")

    # ---------------------------------------------------------------
    # Step 2. Center EACH FEATURE (each row), then compute covariance.
    # ---------------------------------------------------------------
    mean = X.mean(axis=1, keepdims=True)  # (64, 1)
    A = X - mean                          # (64, 1797)
    C = (A @ A.T) / (n - 1)               # (64, 64)

    print("\n2. CENTERING AND COVARIANCE")
    print("Shape of centered matrix A:", A.shape)
    print("Shape of covariance matrix C:", C.shape)
    print("Total sample variance, trace(C):", float(np.trace(C)))
    assert np.allclose(A.mean(axis=1), 0.0)
    assert np.allclose(C, C.T)

    # ---------------------------------------------------------------
    # Step 3. Compute PCA from the SVD: A = U Sigma V.T.
    # ---------------------------------------------------------------
    U, s, Vt = np.linalg.svd(A, full_matrices=False)
    variances = s**2 / (n - 1)
    total_energy = np.sum(s**2)
    ratios = s**2 / total_energy
    cumulative_ratios = np.cumsum(ratios)

    # Choose the SMALLEST k that retains at least 90% of total variance.
    variance_target = 0.90
    k = int(np.searchsorted(cumulative_ratios, variance_target) + 1)
    r = int(np.linalg.matrix_rank(A))
    assert 1 <= k < r

    print("\n3. SVD AND COMPONENT SELECTION")
    print("Numerical rank of A:", r)
    print("First ten singular values:", s[:10])
    print("First ten principal-component variances:", variances[:10])
    print(f"Components required for {variance_target:.0%} variance: {k}")
    print(f"Cumulative explained variance: {cumulative_ratios[k-1]:.4%}")
    assert np.isclose(np.sum(variances), np.trace(C))
    assert np.allclose((U * s) @ Vt, A)

    # ---------------------------------------------------------------
    # Step 4. Reduce from 64 original features to k principal scores.
    # ---------------------------------------------------------------
    Uk = U[:, :k]      # (64, k): LEFT singular vectors
    Z = Uk.T @ A       # (k, 1797): reduced observations in COLUMNS

    # Reconstruct the centered data and restore feature means.
    Ahat = Uk @ Z      # (64, 1797): rank-k approximation to A
    Xhat = Ahat + mean # (64, 1797): reconstructed original digits
    Ak_svd = (U[:, :k] * s[:k]) @ Vt[:k, :]

    print("\n4. PROJECTION AND RECONSTRUCTION")
    print("U_k shape:", Uk.shape)
    print("Z shape (k, observations):", Z.shape)
    print("Reconstructed X shape:", Xhat.shape)
    assert np.allclose(Ahat, Ak_svd)
    assert np.linalg.matrix_rank(Ahat) == k
    # Optional covariance of PCA scores: diagonal with retained variances.
    assert np.allclose((Z @ Z.T) / (n - 1), np.diag(variances[:k]))
```

```
224
225     # ------------------------------------------------------------
226     # Step 5. Verify Eckart--Young in both matrix norms.
227     # ------------------------------------------------------------
228     E = A - Ahat
229     fro_sq = np.linalg.norm(E, ord="fro")**2
230     fro_prediction = np.sum(s[k:]**2)
231     spectral_error = np.linalg.norm(E, ord=2)
232     spectral_prediction = s[k]
233     fraction_discarded = fro_sq / total_energy
234
235     print("\n5. ECKART--YOUNG VERIFICATION")
236     print(f"Actual squared Frobenius error: {fro_sq:.8f}")
237     print(f"Sum of discarded singular values squared: {fro_prediction:.8f}")
238     print(f"Actual spectral error: {spectral_error:.8f}")
239     print(f"Next singular value sigma_(k+1): {spectral_prediction:.8f}")
240     print(f"Fraction of squared error: {fraction_discarded:.4%}")
241     print(f"1 - explained-variance ratio: {1 - cumulative_ratios[k-1]:.4%}")
242     assert np.isclose(fro_sq, fro_prediction)
243     assert np.isclose(spectral_error, spectral_prediction)
244     assert np.isclose(fraction_discarded, 1 - cumulative_ratios[k-1])
245
246     # For comparison, a RANDOM k-dimensional subspace retains less variance.
247     rng = np.random.default_rng(2026)
248     Q_random, _ = np.linalg.qr(rng.standard_normal((m, k)))
249     B_random = Q_random @ (Q_random.T @ A)
250     random_fro_sq = np.linalg.norm(A - B_random, ord="fro")**2
251     print(f"Random subspace squared Frobenius error: {random_fro_sq:.8f}")
252     assert random_fro_sq + 1e-6 >= fro_sq
253
254     # ------------------------------------------------------------
255     # Step 6. Visualize individual and cumulative explained variance.
256     # ------------------------------------------------------------
257     fig, ax = plt.subplots(figsize=(8, 5))
258     pcs = np.arange(1, len(s) + 1)
259     ax.plot(pcs, cumulative_ratios, label="Cumulative explained variance")
260     ax.axhline(variance_target, linestyle="--",
261                label=f"{variance_target:.0%} target")
262     ax.axvline(k, linestyle=":", label=f"Selected k = {k}")
263     ax.set(xlabel="Number of principal components retained",
264            ylabel="Fraction of total variance retained",
265            title="Implementation 2: explained variance for digits")
266     ax.set_ylim(0, 1.04)
267     ax.grid(alpha=0.3)
268     ax.legend()
269     fig.tight_layout()
270     destination = OUTPUT_DIR / "implementation2_variance.png"
271     fig.savefig(destination, dpi=250)
272     plt.show()  # Display each figure now; do not wait until the end.
273     plt.close(fig)
274     print("\nExplained-variance figure saved:", destination)
275
276     # ------------------------------------------------------------
277     # Step 7. Display a 2D PCA projection (for visualization only).
278     # The plotted 2D coordinates use the first TWO components; this
279     # is separate from the k-component reconstruction above.
280     # ------------------------------------------------------------
281     Z2 = U[:, :2].T @ A  # (2, 1797): each observation is a column
282     fig, ax = plt.subplots(figsize=(8, 6))
283     scatter = ax.scatter(Z2[0], Z2[1], c=labels, cmap="tab10",
284                          s=13, alpha=0.65)
285     ax.set(xlabel="Principal component 1 score",
286            ylabel="Principal component 2 score",
287            title="Digits dataset: first two PCA score coordinates")
288     fig.colorbar(scatter, ax=ax, label="Actual digit label")
289     ax.grid(alpha=0.2)
290     fig.tight_layout()
291     destination = OUTPUT_DIR / "implementation2_scores.png"
```

```
    fig.savefig(destination, dpi=250)
    plt.show()  # Display each figure now; do not wait until the end.
    plt.close(fig)
    print("2D score figure saved:", destination)

    # ------------------------------------------------------------------
    # Step 8. Compare original digit images to reconstructed images.
    # We plot one sample for each digit class (0 through 9).
    # Clipping is only for drawing; errors above use UNCLIPPED Xhat.
    # ------------------------------------------------------------------
    chosen = [int(np.flatnonzero(labels == d)[0]) for d in range(10)]
    fig, axes = plt.subplots(2, 10, figsize=(12, 3.3))
    for d, j in enumerate(chosen):
        axes[0, d].imshow(X[:, j].reshape(8, 8), cmap="gray_r",
                          vmin=0, vmax=16)
        axes[1, d].imshow(np.clip(Xhat[:, j], 0, 16).reshape(8, 8),
                          cmap="gray_r", vmin=0, vmax=16)
        axes[0, d].set_title(str(d))
        for ax in (axes[0, d], axes[1, d]):
            ax.axis("off")
    axes[0, 0].set_ylabel("Original")
    axes[1, 0].set_ylabel(f"Rank-{k}")
    fig.suptitle("Original and PCA-reconstructed handwritten digits")
    fig.tight_layout()
    destination = OUTPUT_DIR / "implementation2_reconstructions.png"
    fig.savefig(destination, dpi=250)
    plt.show()  # Display each figure now; do not wait until the end.
    plt.close(fig)
    print("Original/reconstructed digit figure saved:", destination)
    print("All Implementation 2 mathematical checks passed.")


# ============================================================
# RUN BOTH IMPLEMENTATIONS, IN ORDER
# ============================================================
if __name__ == "__main__":
    print("\n" + "=" * 67)
    print("PCA FROM SCRATCH: FEATURES IN ROWS, OBSERVATIONS IN COLUMNS")
    print("=" * 67)
    print("\n" + "-" * 67)
    print("IMPLEMENTATION 1: SMALL DATASET")
    print("-" * 67)
    run_small_example()

    print("\n" + "-" * 67)
    print("IMPLEMENTATION 2: IMPORTED DIGITS DATASET")
    print("-" * 67)
    run_digits_example()
    print("\nBoth implementations completed successfully.")
```

7 Conclusion and Future Directions

In this note, we have attempted to develop Principal Component Analysis from its fundamental mathematical principles, rather than treating it merely as a computational procedure.

We began with a brief review of Singular Value Decomposition, examining its relationship with the row space, column space, singular vectors, and orthogonal projections. We then introduced the geometric motivation behind PCA and explained how variance and covariance allow us to identify the dominant directions of variation in a dataset.

Building upon these ideas, we developed the PCA procedure step by step: centering the obser-

vations, computing the singular value decomposition, selecting the principal directions, projecting the original observations, and reconstructing their low-dimensional approximations.

An important objective of our discussion was to explain not only how these operations are performed, but also why they are mathematically justified.

In particular, we established that the first k left singular vectors identify a subspace that maximizes the total retained variance among all k-dimensional orthogonal projections.

We subsequently showed that maximizing the retained variance is equivalent to minimizing the squared Frobenius reconstruction error.

Finally, through the Eckart–Young theorem, we established a considerably stronger result: the truncated Singular Value Decomposition provides an optimal rank-k approximation among all matrices of rank at most k, under both the Frobenius and spectral norms.

Thus, the approximation

$$\boxed{A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T}$$

is not merely a convenient construction. Its optimality follows from a rigorous mathematical theorem.

We also implemented PCA using Python, first on a small, manually constructed dataset and subsequently on a larger dataset of handwritten digit images. These experiments demonstrated how dimensionality reduction and reconstruction work in practice, while numerically verifying the error formulas predicted by the Eckart–Young theorem.

Nevertheless, despite its mathematical elegance and widespread usefulness, PCA is far from a perfect method, and the practical use of SVD is not without limitations.

One important difficulty concerns categorical and mixed-type datasets. Ordinary PCA is based on numerical coordinates, Euclidean distances, and variance. When categorical variables are represented by arbitrary numerical labels, these operations may introduce artificial relationships between categories.

Even when categorical observations are converted into numerical vectors through methods such as one-hot encoding, the resulting Euclidean geometry may not adequately reflect the underlying structure of the data.

It is important to emphasize that this is primarily a limitation of ordinary PCA and the way the data is represented, rather than a failure of the SVD theorem itself.

Methods such as Multiple Correspondence Analysis and techniques designed specifically for mixed numerical and categorical data provide alternative approaches in such situations.

Another important challenge arises in large-scale scientific computing.

Although highly reliable numerical algorithms for computing SVD already exist, calculating a full decomposition of an extremely large matrix can require substantial computational time and

memory.

This becomes particularly significant in applications involving high-resolution images, large scientific simulations, massive datasets, or continuously arriving observations.

Moreover, PCA identifies linear subspaces and maximizes variance, which may not capture nonlinear relationships or necessarily preserve the information most relevant to a particular predictive task.

Fortunately, these limitations have also motivated considerable progress in numerical linear algebra and machine learning.

Research on randomized and truncated SVD algorithms, matrix sketching, incremental PCA, distributed computing, and hardware-accelerated numerical methods continues to improve the scalability of low-rank approximation.

At the same time, nonlinear dimensionality-reduction techniques, methods for categorical and mixed data, and more flexible representation models offer promising alternatives when the assumptions of classical PCA are inappropriate.

These developments suggest that the future of dimensionality reduction will not necessarily consist of replacing SVD, but rather of extending its computational reach and combining its mathematical strengths with methods better suited to increasingly complex forms of data.

We may therefore expect further improvements in the efficiency, scalability, and applicability of low-dimensional approximation methods in the coming years.

We conclude by emphasizing that understanding the limitations of a mathematical method is just as important as understanding its strengths.

The Eckart–Young theorem tells us precisely when and why truncated SVD is optimal. It does not claim that low-rank approximation is the appropriate objective for every possible dataset.

Recognizing this distinction allows us to appreciate both the remarkable mathematical beauty of PCA and the important challenges that remain.

Our hope is that this exposition has helped the reader see PCA not as an obscure black-box algorithm, but as a natural and elegant consequence of elementary linear algebra, geometry, and optimization, while also encouraging further exploration of the rapidly evolving field of numerical data analysis.

References

[1] G. Strang, *Linear Algebra and Its Applications*, 4th ed., Cengage Learning, 2006.

[2] J. N. Kutz, *Data-Driven Modeling & Scientific Computation: Methods for Complex Systems & Big Data*, Oxford University Press, 2013.

[3] S. J. Belhaiza, *Principal Component Analysis*, MATH 557 video lecture, King Fahd University of Petroleum and Minerals (KFUPM). Available online: https://www.youtube.com/watch?v=lpBCHV1rQTI.

[4] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed., Springer Series in Statistics, Springer, New York, 2002. https://doi.org/10.1007/b98835.

[5] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, Society for Industrial and Applied Mathematics (SIAM), 1997.

[6] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.

[7] C. Eckart and G. Young, "The Approximation of One Matrix by Another of Lower Rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936. https://doi.org/10.1007/BF02288367.

[8] L. Mirsky, "Symmetric Gauge Functions and Unitarily Invariant Norms," *The Quarterly Journal of Mathematics*, vol. 11, no. 1, pp. 50–59, 1960. https://doi.org/10.1093/qmath/11.1.50.

[9] N. Halko, P. G. Martinsson, and J. A. Tropp, "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions," *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011. https://doi.org/10.1137/090771806.